International Journal of Control, Automation, and Systems VV(X) (YYYY) 1-9
http://dx.doi.org/10.1007/s12555-xxx-xxxx-x
ISSN:1598-6446 eISSN:2005-4092
http://www.springer.com/12555

Learned Direct Actuation versus Tuned and RL-Tuned Pure Pursuit: A Controlled Comparison for Path Tracking

Dileep Banothu and N. Sanjay Varma*

Abstract

Two parameters govern Pure Pursuit (PP) tracking, its steering gain and its lookahead distance, and reinforcement learning (RL) has recently been integrated with PP by letting an agent tune both online. We compare such integration of learning with classical control against learning that replaces the geometric law. A network drives a "bead" (a point mass) by choosing its acceleration along references extracted from images and is trained by imitation (DAgger), by PPO (Proximal Policy Optimization), or by both. It is compared with tuned PP, an adaptation of RL-tuned PP, and the PP law run on the bead, under one simulator and lap rule, on eight held-out F1TENTH circuits with five training seeds. Imitating a hand-written rule, the bead matches that rule (1.06 cm RMSE). With our PPO budget and settings, PPO did not improve this imitation policy. Tuned PP reaches 0.31 cm at the bead's speed, and 0.28 cm when run on the bead, so the gap stems from the control law, not the vehicle; against the true centerlines it shrinks from 0.75 to 0.36 cm but keeps its sign. Imitating the PP law instead, the network reaches 0.42 cm and matches tuned PP against the true centerlines. On a dynamic single-track car, gains must be re-tuned; PP then stays the most accurate car controller (1.83 cm), its lead over Stanley being clear with tyres only and unclear with a steering-rate limit. A naive protocol with test leakage and untuned baselines would have reversed these conclusions.

Keywords Pure pursuit · Path tracking · Imitation learning · Reinforcement learning · F1TENTH

1 Introduction

Turning a planned path into commands is the job of the path tracker in an autonomous vehicle. On small racing cars like F1TENTH [1] this is usually a geometric controller, most often Pure Pursuit (PP) [2, 3], which steers toward a point a lookahead distance L_d ahead on the path with a gain g. A short lookahead reacts quickly but weaves; a long one is smooth but cuts corners.

Computational intelligence can be integrated with such a controller in two ways. In RL-tuned PP [4], an agent trained with Proximal Policy Optimization (PPO) [5] selects (L_d, g) online while the PP law stays in the loop—learning *inside* classical control. The alternative replaces the law: a network outputs the actuation itself and can be trained by imitation of a teacher, by reinforcement learning (RL), or both. Whether the second route is more precise is rarely tested under identical conditions.

We run that test with a "bead", a point mass following references extracted from images, and design the comparison against three confounds. Test leakage and untuned baselines can make a learned controller look better than it is, so the test circuits stay out of training, selection and tuning, every baseline is tuned until its optimum lies inside the searched range, and the imitated teacher is reported. The vehicle can matter as much as the controller—the bead accelerates in any direction, the PP car must steer—so both plants share one lap rule and the PP law is also run on the bead. And the extracted reference has its own error, so all results are also measured against the true circuit centerlines.

The building blocks are standard. Our contribution is the controlled comparison and its findings:

- a comparison of a direct-actuation network trained by DAgger [6], by PPO and by DAgger-initialized PPO with tuned PP and an adaptation of RL-tuned PP [4], with one simulator, one lap rule and one metric code, at native and matched speed;
- same-plant controls—the PP law on the bead, the expert aiming further ahead, and DAgger with the PP law as teacher—that separate the control law, the vehicle and the teacher;
- the finding that, on the kinematic car and on the bead, a well-tuned PP law is more accurate than every learned controller that does not imitate it; that the imitation policy is limited mainly by its teacher, matching tuned PP on the real track only when taught by the PP law; and that, with our budget and settings, PPO did not improve it;

Dileep Banothu and N. Sanjay Varma are with the Indian Institute of Information Technology Sri City, Sri City, Andhra Pradesh 517646, India (e-mails: [D. Banothu e-mail], sanjayvarma.n24@iiits.in).
* Corresponding author.

©ICROS, KIEE and Springer YYYY

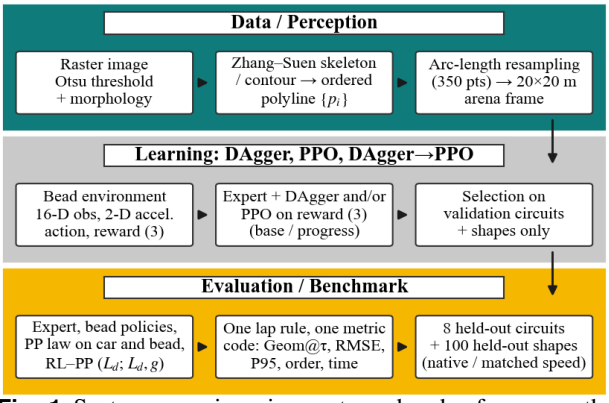


Fig. 1 System overview: image to ordered reference path; bead network trained by DAgger, PPO or DAgger→PPO with selection on validation paths; all controllers scored with one lap rule and one metric code on circuits and shapes kept out of training.

- a tuned Stanley baseline, and a dynamic single-track check at matched speed where the geometric laws are re-tuned and RL–PP is retrained;
- a naive-protocol contrast that shows how test leakage and untuned baselines would have reversed these conclusions.

2 Related Work

Geometric trackers. Pure Pursuit [2] and the Stanley method [7] are the most familiar geometric trackers [3]; their vehicle models are covered in [8, 9]. Many PP variants adapt L_d to speed or curvature with hand-chosen formulas. Racing lines often come from an optimization that minimizes curvature [10, 11].

Learned control. Deep RL is widely used for driving [12] and robotics [13]. PPO [5], paired with GAE (Generalized Advantage Estimation) [14], is a standard on-policy method whose results depend on many implementation choices [15]; off-policy methods like SAC [16] are more sample-efficient. End-to-end networks are flexible but hard to interpret and transfer [17]; tuning a classical controller with RL [4] integrates both.

Imitation learning. Behaviour cloning goes back to ALVINN [18]; DAgger [6] counters compounding errors by labelling the states the learner visits. An imitation learner approaches its teacher, and going beyond it normally calls for RL fine-tuning. We report the teacher in every comparison, and vary it, to separate what the network learned from what the teacher supplies.

3 Methods

Fig. 1 outlines the system.

3.1 Reference extraction

Images are thresholded with Otsu's method [19], cleaned by a 3×3 closing, and reduced to a skeleton one pixel wide with the Zhang–Suen algorithm [20] (bigger regions are followed along their boundary). Each skeleton piece is ordered by a walk from one end, scaled into a 20×20 m arena, and given about 350 equally spaced points; images without a valid path are reported as failures. Each circuit image is drawn from a centerline we know, which makes the extraction error directly measurable: per circuit, the extracted reference lies on average 0.79–1.24 cm from the source centerline, and at most 6.53 cm. Rasterization also shrinks the F1TENTH tracks by a linear factor of 4.3–9.8 (median 6.5), to 39–82 m, and the spacing of the extracted waypoints is 11.1–23.5 cm per circuit (median 16.6 cm; supplementary Table S1). Four example inputs and their extracted paths appear in supplementary Fig. S2.

3.2 Bead dynamics, observation and reward

The bead is modelled as a point mass with position $\mathbf{p}_t$, velocity $\mathbf{v}_t$ and command $\mathbf{u}_t \in [-1,1]^2$, scaled to at most $a_{\max} = 10$ m/s^2 per axis, with viscous friction $\mu = 7$ s^{-1}, $v_{\max} = 7$ m/s and $\Delta t = 0.016$ s:

$$\begin{aligned} \mathbf{v}_{t+1} &= (\mathbf{v}_t + a_{\max}\mathbf{u}_t\Delta t)(1-\mu\Delta t), \\ \mathbf{p}_{t+1} &= \mathbf{p}_t + \mathbf{v}_{t+1}\Delta t. \end{aligned} \tag{1}$$

Full actuation along a single axis yields the steady speed $v^* = a_{\max}\Delta t(1-\mu\Delta t)/(\mu\Delta t) \approx 1.27$ m/s. A waypoint counter w_t may move ahead by four indices or fewer in one step, and only onto points closer than $\rho = 0.18$ m; a lap is finished when w_t reaches 99% of the path, and episodes end after $\max(3500, \lceil 1.5L/(v^*\Delta t)\rceil)$ steps for a path of length L or after 400 steps without progress. The PP car uses the same counter code, lap rule, stall rule and step cap.

The policy observes 16 values scaled to $[-1,1]$ by construction (vectors to waypoints w_t, w_t+1 and w_t+5, position, velocity and heading, the distance to the path, an on-path flag, a curvature measure and the completed fraction), so no running input normalization is used. With e_t the gap to the closest point of the reference, d_t the gap to waypoint w_t, n_t the waypoints passed in step t and $\hat{\mathbf{r}}_t$ the path tangent, the reward is

$$\begin{aligned} r_t = {} & 0.1\,\mathrm{clip}\big((d_{t-1}-d_t)/\rho, \pm 1\big) + 0.4\,n_t \\ & + 0.25\,e^{-(e_t/0.08)^2} + 0.15\,\mathrm{clip}(\mathbf{v}_t\cdot\hat{\mathbf{r}}_t/v_{\max}, \pm 1) \\ & - 0.05\min(e_t/0.1, 3) - 0.1\cdot\mathbf{1}[e_t > \rho] \\ & - 0.05\cdot\mathbf{1}_{\text{back}} - 0.5\cdot\mathbf{1}_{\text{wall}} \\ & - 0.005\min(\|\mathbf{u}_t - \mathbf{u}_{t-1}\|, 1) - 0.001, \end{aligned} \tag{2}$$

plus $+50$ for a finished lap, $+15$ for a truncated episode past 80% and -5 otherwise; the weights were set before

the data split and not tuned. Because every step pays the centering bonus, a slower lap earns more of it: adding Gaussian noise of standard deviation 0.61 to the expert's actions triples its validation RMSE (0.91 to 2.92 cm) yet raises its return from 746 to 764. A *progress* variant therefore pays the bonus per waypoint passed, $2.0\,n_t e^{-(e_t/0.08)^2}$, keeping its per-lap budget at the expert's pace; the noisy expert then earns less than the noise-free one (725 versus 836).

3.3 Learning the bead controller

The policy is an actor–critic from Stable-Baselines3 [21]; actor and critic are separate tanh networks, each having two hidden layers, 256 units wide, and its Gaussian spread independent of the state; evaluation applies the mean action. The hand-written *expert* steers toward the next waypoint,

$$\mathbf{u}_t^* = (\mathbf{p}[w_t+1] - \mathbf{p}_t)/\|\mathbf{p}[w_t+1] - \mathbf{p}_t\|. \tag{3}$$

Bead-DAgger first regresses the actor on a single teacher episode for each training path and refines it with DAgger (Algorithm 1); the reward is not used. The teacher is the expert (3) or, in *Bead-DAgger (Bead-PP teacher)*, the PP law run on the bead (Section 3.5).

Bead-PPO starts from random weights and maximizes PPO's clipped objective,

$$\begin{aligned}\mathcal{L}(\theta) = \mathbb{E}_t\big[\min(\rho_t\hat{A}_t, \mathrm{clip}(\rho_t, 1\pm\varepsilon)\hat{A}_t)\big] \\ - c_v\mathcal{L}^V + c_e\mathcal{H},\end{aligned} \tag{4}$$

with GAE advantages [14], for 3×10^6 steps. The starting value $\log\sigma_0 = -0.5$ of the policy's log standard deviation was picked among $\{-1,-0.5,0\}$ by a pilot with seed 100 scored on validation references only (scores 0.750, 0.851 and 0.811 after 7.5×10^5 steps).

Bead-DAgger→PPO starts from the expert-taught DAgger actor with the same seed, holds that actor fixed for 5×10^4 steps while only the critic learns, and continues with (4) for 2×10^6 steps. To avoid scrambling the initial policy it starts with $\log\sigma_0 = -2$, uses the learning rate 10^{-4}, and adds a behaviour-cloning term

$$\mathcal{L}_{\mathrm{BC}}(\theta) = \lambda\,\mathbb{E}_t\big[\|\boldsymbol{\mu}_\theta(\mathbf{o}_t) - \mathbf{u}_t^*\|^2\big], \quad \lambda = 1, \tag{5}$$

where $\boldsymbol{\mu}_\theta$ is the actor mean and $\mathbf{u}_t^*$ is recovered exactly from the observation. Both PPO procedures are run with the base and the progress reward.

Each variant learns on the training split, and its final checkpoint is the one scoring highest on validation; the score is average Geom@0.10 (as a fraction) less average RMSE (in meters), averaged over validation circuits and shapes. In these units 1 cm of RMSE weighs as much as one percentage point of coverage; RMSE alone does not notice skipped sections, which Geom@0.10 penalizes. Selecting by RMSE alone would have chosen the same checkpoint or setting in 32 of 45 runs and grids, and otherwise a validation RMSE within 0.48 cm. Hyperparameters are listed in Table 1.

Table 1 Learning hyper-parameters.

Quantity	Bead (PPO variants)	RL–PP (adaptation of [4])
MLP (policy/value)	[256, 256] tanh	[64, 64] tanh
n_{steps} / batch / epochs	2048×4 envs / 128 / 10	2048 / 256 / 5
γ / GAE λ / clip	0.99 / 0.95 / 0.2	0.99 / 0.98 / 0.2 (KL 0.015)
Entropy / value coeff.	0.001 / 0.5	0.02 / 0.6
Learning rate	3×10^{-4}; fine-tune 10^{-4}	2.4×10^{-4}, linear decay
Initial $\log\sigma$	-0.5; fine-tune -2	0
BC weight λ	fine-tune 1 (check: 0.1, 10)	–
Normalization	none (inputs in $[-1,1]$)	VecNormalize (obs., return)
Steps per run	3×10^6; fine-tune 2×10^6	10^5
Training seeds	0–4	0–4

Algorithm 1 DAgger for the bead (Bead-DAgger).

1: **Input:** training paths $\mathcal{T}$, validation paths $\mathcal{V}$, teacher π^*, network π_θ, $\beta = (0.4, 0.2, 0.1, 0.05, 0, 0, 0, 0)$
2: $\mathcal{D} \leftarrow$ one teacher episode per $\tau \in \mathcal{T}$; fit actor on $\mathcal{D}$ (12 epochs, lr 2×10^{-4})
3: **for** round $k = 1, \ldots, 8$ **do**
4: for each $\tau \in \mathcal{T}$ roll out one episode; *at every step* execute π^* with probability β_k, else π_θ, and label the state with $\pi^*(o)$
5: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(o, \pi^*(o))\}$; refit actor (8 epochs, lr 1.5×10^{-4} for $k \le 3$, else 7×10^{-5})
6: keep the checkpoint if the validation score on $\mathcal{V}$ improves
7: **end for**
8: **return** best validation checkpoint

3.4 Pure Pursuit baselines

The PP car is a kinematic bicycle [8] with wheelbase $W = 0.33$ m. With y' the sideways offset, in the car's frame, of the lookahead point,

$$\begin{aligned}&\gamma = g\cdot 2y'/L_d^2, \qquad \dot{\theta} = (v/W)\tan\delta, \\ &\delta = \mathrm{clip}(\arctan(W\gamma), \pm 1.10\ \mathrm{rad}).\end{aligned} \tag{6}$$

Speed follows $v_{\mathrm{ref}} = \mathrm{clip}(4.2/\sqrt{1+2.5|\kappa|}, 1.8, 5.0)$ m/s with braking and acceleration passes. The model has no tyre forces: at native speed, tuned PP demands more than 10 m/s^2 of lateral acceleration in 12.4% of its test steps, which a real car could not sustain; at the matched speed below, the maximum is 9.4 m/s^2.

Fixed and Adaptive PP use constant (L_d, g) and $L_d = \mathrm{clip}(a + bv, 0.05, 1.20)$ with $g = 1.05$. Both are tuned by grid search over the training references, using the selection score above, over $L_d \in \{0.05, 0.075, \ldots, 0.20\} \cup \{0.25, 0.30, \ldots, 1.00\}$ m, $g \in \{0.6, \ldots, 1.4\}$, $a \in \{0, 0.025, \ldots, 0.5\}$ and $b \in \{0, 0.025, 0.05, \ldots, 0.25\}$ s. Fixed PP selects (0.125 m, 0.9) at native and (0.125 m, 1.0) at the matched speed, inside the grid (training RMSE 0.37, 0.24 and 0.26 cm at 0.10, 0.125 and

Table 2 RL-tuned PP: original [4] and our adaptation.

Aspect	[4]	This work
Track scale	F1TENTH maps	shrunk 4.3–9.8×, 20×20 m
L_d / g range	$[0.35, 4.0]$ m / $[0.45, 1.15]$	$[0.05, 1.20]$ m / $[0.60, 1.40]$
Teacher L^*	$0.50 + 0.28v - 3.5\kappa_{\max}$	$0.65 + 0.04v - 0.18\kappa_{\max}$
Teacher g^*	decreasing in v	increasing in $\kappa_{\max}$
Observations	$v, \kappa_{0,1,2}, \Delta\kappa$	+ lateral and heading error
Reward	11 weighted terms	own weights; no $\lvert\kappa\rvert$ term; + corridor, error, alignment terms
Speed profile	raceline, $\leq$15.6 m/s	friction-limited, 1.8–5 m/s
Steering limit	PP command ± 0.35	$\lvert\delta\rvert \leq 1.10$ rad
Training data	Hockenheim	11 circuits + 120 contours
Budget	$\leq 1.2 \times 10^6$ steps, Optuna	10^5 steps, no tuning
Evaluation	simulation + 1:10 car	simulation only

0.15 m). Adaptive PP selects $(a, b) = (0.05, 0.025)$, but the best constant lookahead scores within 0.001 cm, so it is in effect a Fixed PP.

Stanley [7] runs on the same car: $\delta = \psi_{\text{path}} - \psi + \arctan\big(k e_f/(k_s + v)\big)$, clipped to ± 1.10 rad, where e_f is the signed distance of the front axle from the nearest path segment and ψ_{path} that segment's heading. Its gains are grid-searched with the same references, score and completion rule as PP, over $k \in \{0.25, \ldots, 4096\}$ and $k_s \in \{0.005, \ldots, 4\}$ m/s (220 settings; smaller grids had their optimum on the edge and were widened until it lay inside). It selects $(k, k_s) = (512, 0.01)$ at native and $(1024, 1)$ at the matched speed, both inside the grid; from $k = 512$ upward the score changes by less than 0.001.

RL–PP adapts [4], an arXiv preprint that has not been peer reviewed, to our arena. A PPO agent observes speed, the curvature at the current waypoint and at those 5 and 12 ahead, their difference, the lateral error and the heading error, and outputs $L_d \in [0.05, 1.20]$ m and $g \in [0.60, 1.40]$, low-pass filtered with factor 0.2. Its reward follows the structure of [4]: distance from a teacher $L^* = \text{clip}(0.65 + 0.04v - 0.18\kappa_{\max}, 0.26, 0.85)$, $g^* = \text{clip}(0.96 + 0.14\kappa_{\max}, 0.6, 1.4)$, parameter-change penalties, a bend term, progress, speed, wall and stall terms, plus a corridor bonus, a quadratic error penalty and an alignment bonus. Table 2 lists every change; a separately trained L_d-only agent ($g = 1.05$) serves as an ablation.

3.5 Pure Pursuit on the bead

Bead-PP runs the car's Fixed PP law on the bead. With the same nearest-point window and lookahead search as the car and the heading taken from the bead's velocity, the curvature γ of (6) is applied as a lateral acceleration $v^2\gamma$; a longitudinal acceleration, obtained by inverting one step of (1), holds the speed at v_{match}, and steering has priority when $a_{\max}$ is reached. Bead-PP is tuned on the same grid and references as the car and selects $(0.125\text{ m}, 1.2)$, inside the grid. As a second control, the expert (3) aims at waypoint $w_t + k$, $k = 1, \ldots, 10$, anchored on w_t or on the tracked nearest point; the best k is selected on training references.

4 Experimental Setup

4.1 Data, protocol and metrics

Every one of the 23 circuits in the F1TENTH racetrack collection[1] were rasterized and extracted. Yas Marina, Hockenheim and Montreal go to the test split; a seeded permutation chosen ahead of training divides the remaining 20 into 11 training, 4 validation and 5 additional test circuits (supplementary Fig. S1). Two synthetic image sets give training 60 closed and 60 open contours, 25 references per set to checkpoint selection, and 50 per set to testing. In total 493 references were extracted without failure.

Each controller drives every test circuit five times (start perturbed by up to ± 5 cm) and every test contour once, deterministically. Learned controllers have five training seeds (three in the behaviour-cloning weight check). In Track A each controller drives at its own pace; in Track B the car is commanded no faster than $v_{\text{match}} = 1.256$ m/s, the expert's mean speed on the training circuits, and PP is tuned and RL–PP trained again at this speed; Bead-PP runs at v_{match}. Counting training seeds separately, 60 controller instances ran 8400 test episodes. Every metric was recomputed from the stored trajectories, partly with independent code, and re-running 22 instances from their saved checkpoints reproduced the trajectories bit for bit.

All distances are to *polylines*, not to points. With $\mathcal{P}$ the set of reference points, P the polyline through them and X the trajectory polyline, the tracking error of sample $\mathbf{x}_t$ is $e(\mathbf{x}_t) = \text{dist}(\mathbf{x}_t, P)$, which yields the RMSE, the largest error and the error's 95th percentile (P95); coverage counts the reference points lying at most τ from the trajectory polyline,

$$\text{Geom@}\tau = \frac{1}{|\mathcal{P}|} \sum_{\mathbf{p} \in \mathcal{P}} \mathbf{1}\big[\text{dist}(\mathbf{p}, X) \leq \tau\big]. \tag{7}$$

Ordered coverage is the size of the biggest group of reference points reached ($\leq \rho$) in path order, computed by dynamic programming; a run with ordered coverage of at least 0.95 is completed. The same code scores every controller, against the extracted reference and, for the circuits, against the true centerline. Under a *naive protocol* we also score a bead fitted to the expert by regression, with its checkpoint chosen on Montreal, Yas Marina and Hockenheim, an RL–PP agent trained on just one hand-drawn contour and run without its normalization statistics, and PP with hand-set gains $(0.55\text{ m}, 1.05)$ and $(a, b) = (0.35, 0.25)$; these rows enter no comparison.

Statistics. Every comparison pairs the two controllers circuit by circuit on the 8 test circuits, after averaging seeds and episodes. The main evidence is the mean difference and its bootstrap interval at the 95% level (10 000

[1] `https://github.com/f1tenth/f1tenth_racetracks`, commit b95c4ef.

Table 3 Accuracy against the extracted references on the eight unseen test circuits, averaged over circuits with five episodes per circuit. Learned controllers: average of five training seeds (three for $\lambda = 10$), with the seed range in brackets. Bold: most accurate entry of each column.

Controller	Geom@0.05 [%]	Geom@0.10 [%]	RMSE [cm]	P95 [cm]	Max [cm]	Ordered [%]	Completed [%]	Time [s]	Speed [m/s]
Bead plant (holonomic point mass, damping-limited)									
Geometric expert (3)	98.6	**99.2**	1.07	2.55	5.02	**99.1**	100	42.5	1.25
Bead-PP (PP law on bead)[§]	**99.0**	**99.2**	**0.28**	0.51	**2.72**	**99.1**	100	42.5	1.25
Bead-DAgger	98.4	99.1 [99.1, 99.1]	1.06 [1.05, 1.07]	2.47	5.47	**99.1**	100	42.4	1.25
Bead-DAgger, Bead-PP teacher	**99.0**	**99.2** [99.2, 99.2]	0.42 [0.41, 0.43]	0.83	3.19	**99.1**	100	42.8	1.24
Bead-DAgger→PPO	96.6	99.1 [99.0, 99.1]	1.77 [1.51, 2.14]	3.56	7.39	**99.1**	100	42.2	1.25
Bead-DAgger→PPO, $\lambda = 10$ (3 seeds)	98.2	99.1 [99.1, 99.1]	1.26 [1.22, 1.32]	2.86	5.99	**99.1**	100	42.7	1.24
Bead-DAgger→PPO, prog. rew.	95.0	98.9 [98.8, 99.1]	2.14 [1.75, 2.31]	4.42	8.64	**99.1**	100	40.9	1.30
Bead-PPO (scratch)	69.5	89.8 [82.6, 94.8]	5.12 [4.25, 5.96]	10.64	19.27	96.4	98	36.8	1.36
Bead-PPO (scratch), prog. rew.	59.0	83.1 [76.8, 86.0]	6.49 [6.00, 7.73]	13.63	22.29	95.9	74	37.0	1.38
Bead, naive protocol*	77.0	93.9	4.38	9.26	16.30	98.9	100	42.5	1.23
Bicycle plant, native speed profile (Track A)									
Fixed PP (hand-set gains)	85.0	94.4	3.85	8.52	18.98	98.0	100	18.2	2.90
Fixed PP (tuned)	**99.0**	**99.2**	0.34	0.57	3.26	**99.1**	100	18.5	2.87
Adaptive PP (hand-set)	24.8	33.7	10.12	22.97	32.55	40.8	0	8.0	2.89
Adaptive PP (tuned)	98.9	**99.2**	0.42	0.69	3.83	**99.1**	100	18.5	2.87
Stanley (tuned)	80.7	91.9	4.12	9.24	16.82	96.3	88	17.6	2.91
RL–PP (joint L_d, g) [4]	94.2	96.6 [91.5, 99.1]	1.83 [1.27, 2.65]	3.89	11.35	97.0	96	18.1	2.86
RL–PP (L_d only)	97.7	99.0 [98.9, 99.1]	1.57 [1.21, 2.16]	3.23	10.14	**99.1**	100	18.7	2.86
RL–PP, naive protocol†	83.8	95.5	3.68	7.89	16.73	98.6	100	17.9	2.89
Bicycle plant, speed capped at $v_{\text{match}} = 1.256$ m/s (Track B)									
Fixed PP (tuned)	**99.0**	**99.2**	0.31	0.51	3.26	**99.1**	100	42.2	1.26
Adaptive PP (tuned)	98.9	**99.2**	0.35	**0.49**	3.74	**99.1**	100	42.3	1.26
Stanley (tuned)	77.6	88.7	3.56	8.65	16.87	92.6	88	39.3	1.26
RL–PP (joint L_d, g)	93.6	95.6 [89.8, 99.0]	2.40 [1.61, 4.10]	4.75	15.19	96.0	94	41.3	1.26

*Naive protocol: actor regressed onto the expert and selected on three of the eight test circuits (Yas Marina, Montreal, Hockenheim).
†Naive protocol: trained on just one hand-drawn contour; run on raw observations. §The Fixed PP law executed on the bead at v_{match}.

resamples of circuits, and of seeds within learned controllers). With $n = 8$, an exact signed-rank Wilcoxon test cannot give $p < 0.0078$, which requires all circuits to agree, so only four primary RMSE comparisons are tested, with Holm correction: (P1) Bead-DAgger versus Fixed PP at v_{match}; (P2) Bead-DAgger→PPO versus Bead-DAgger; (P3) RL–PP joint versus Fixed PP; (P4) Bead-PP versus the expert. They were declared before the final evaluation; a preliminary run of Bead-PP and of the expert sweep had, however, touched the test circuits before the settings were fixed (no setting was chosen from it). Other comparisons are exploratory. Episodes are not pooled, since those on one circuit differ only by the start perturbation.

5 Results and Discussion

5.1 Accuracy on held-out circuits

Table 3 gives the results against the extracted references. **Bead-DAgger matches its expert** (1.06 versus 1.07 cm) with almost no spread across seeds. All PPO-trained variants are less accurate: Bead-DAgger→PPO reaches 1.77 cm (2.14 cm with the progress reward) while scratch-trained Bead-PPO reaches 5.12 cm (6.49 cm), completing 98% (74%) of its test runs. **Against these references, tuned Fixed PP is more accurate than every learned bead controller**, including the one taught by PP (Section 5.2): it reaches 0.34 cm at native speed, 0.31 cm at v_{match}, and keeps 99.8% of its samples no further than 3 cm from the reference, compared with 96.9% for the expert and 92.7% for Bead-DAgger→PPO (supplementary Figs. S3–S4). RL–PP reaches 1.83 cm (joint) and 1.57 cm (L_d only). Because both plants share the lap rule, Geom@0.10 is equal for every controller that completes its laps. Hand-set Adaptive PP completes no lap: its lookahead of 0.8–1.6 m cuts corners by more than ρ. **Tuned Stanley is less accurate than PP at the car's reference point**: 4.12 cm at native and 3.56 cm at the matched speed, behind Fixed PP on every circuit (3.25 cm [2.88, 3.54] at v_{match}; 2.67 cm against the true centerlines), and it completes 88% of its runs: at one Montreal hairpin its rear axle passes about 20 cm from the path, beyond the lap counter's radius ρ, and the run stops after 400 steps without progress. This follows from what each law regulates: Stanley steers the front axle onto the path, so the rear axle cuts corners by about $W^2/(2R)$ for a corner of radius R, whereas PP steers the rear axle. At the front axle the order reverses (2.22 cm for Stanley, 4.83 cm for Fixed PP at v_{match}). The car's position and all metrics refer to the rear axle, so PP is the matching geometric law here, and our results on tuned geometric control refer to PP.

Measured against the true centerlines (Table 4), every error grows by the extraction error, and the differences between good controllers shrink. The gap between Bead-DAgger and Fixed PP at v_{match} falls from 0.75 [0.62, 0.87] cm to 0.36 [0.22, 0.51] cm, and that between Bead-PP and the expert from -0.79 to -0.36 cm. These gaps are of the size of the extraction error itself (0.79–1.24 cm), but they keep their sign on all eight circuits, all four pri-

Table 4 RMSE against the extracted references and RMSE, P95 and Geom@0.10 against the true F1TENTH centerlines (eight test circuits).

Controller	Extracted RMSE [cm]	True centerline RMSE [cm]	P95 [cm]	G@0.10 [%]
Geometric expert (3)	1.07	1.60	3.16	**99.2**
Bead-PP (PP law on bead)	**0.28**	1.24	1.99	**99.2**
Bead-DAgger	1.06	1.60	3.15	99.1
Bead-DAgger, Bead-PP teacher	0.42	1.24	2.03	**99.2**
Bead-DAgger→PPO	1.77	2.26	4.39	99.0
Bead-PPO (scratch)	5.12	5.49	11.38	88.8
Fixed PP (tuned)	0.34	1.25	2.04	**99.2**
Fixed PP (tuned) @v_{match}	0.31	**1.23**	**1.98**	**99.2**
Adaptive PP (tuned)	0.42	1.28	2.10	**99.2**
Stanley (tuned)	4.12	4.44	9.81	91.6
Stanley (tuned) @v_{match}	3.56	3.90	9.24	88.2
RL–PP (joint L_d, g)	1.83	2.22	4.47	96.5

mary intervals still exclude zero, and the ranking of all controllers is nearly unchanged (rank correlation 0.993).

5.2 Control law, vehicle and teacher

The control law, not the vehicle, explains the gap. On the bead at v_{match} the PP law reaches 0.28 cm, as accurate as on the equally fast car and below the expert on all eight circuits (P4). On the training circuits its error depends on L_d in the same way on both plants, with the same optimum (Fig. 2, left), so the bead is not harder to steer accurately. Aiming the expert further ahead does not help (Fig. 2, right): the best target is the next waypoint ($k = 1$, 1.07 cm), or $k = 2$ with the nearest-point anchor (1.15 cm), and from $k = 4$ on laps are no longer completed. Unlike PP, the expert does not turn the offset of its target into a curvature command.

The teacher, more than the network, limited the imitation policy. Taught by Bead-PP instead of the expert, with the same Algorithm 1, Bead-DAgger reaches 0.42 cm: −0.64 cm [−0.77, −0.48] relative to the expert-taught policy, better on all eight circuits. Against the extracted references it stays 0.15 cm [0.11, 0.18] behind its teacher and 0.12 cm [0.08, 0.15] behind tuned PP on the equally fast car, on every circuit. Against the true centerlines both differences vanish: 0.00 cm [−0.03, 0.04] relative to its teacher and 0.01 cm [−0.03, 0.05] relative to tuned PP (Table 4). The network can therefore represent a controller as accurate as tuned PP on the real track, but by imitation it approaches, and does not exceed, its teacher; it reaches that accuracy only when the PP law itself is the teacher.

5.3 How PPO changes the bead policy

PPO changes the actor, the critic and the action noise (unit tests; with the actor frozen for the critic's warm-up the approximate KL divergence was exactly zero); at the

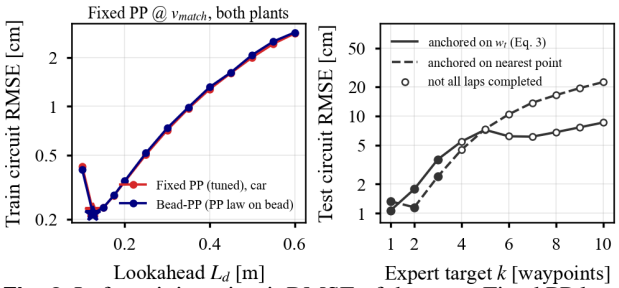


Fig. 2 Left: training-circuit RMSE of the same Fixed PP law on the car and on the bead at v_{match} versus L_d (best gain per L_d; star: selected). Right: test-circuit RMSE of the expert aiming k waypoints ahead; open markers: not every lap completed.

end of training the approximate KL per update is 0.010–0.031 and the explained variance 0.86–0.98. PPO from scratch improves slowly and remains far from the expert after 3×10^6 steps (Fig. 3). Fine-tuning moves away from the DAgger policy despite the small initial noise, the lower learning rate and the behaviour-cloning term, and raised the test RMSE on all eight circuits for both rewards (P2). A validation diagnostic (four circuits, three noise seeds, five training seeds) shows that fine-tuning also lowered the return of the stochastic policy it optimizes—from 749 to 729 with the base reward and from 829 to 804 with the progress reward—so the drop in accuracy cannot be explained by training a noisy policy and testing a noise-free one. With the progress reward the bead became faster and less accurate. A check of the behaviour-cloning weight (seeds 0–2, base reward, selection on validation only) gave mean validation RMSEs of 2.18, 1.57 and 1.09 cm for $\lambda = 0.1$, 1 and 10, against 0.86 cm for the DAgger policy. With the selected $\lambda = 10$, the fine-tuned policy reaches 1.26 cm on the eight test circuits: less degraded than with $\lambda = 1$, but still 0.20 cm [0.15, 0.26] behind its starting point. A stronger pull toward the expert limits the damage but does not turn it into an improvement. Which factor—the critic, the mix of long circuits and short contours, the step size or the budget—causes the degradation was not isolated: **with our PPO budget and settings, PPO did not improve the imitation policy**.

5.4 Naive protocol, speed and contours

On its own training circuits (Hockenheim, Montreal, Yas Marina) the naive bead reaches 2.45–4.05 cm, against 3.62–5.54 cm for PP with hand-set gains and 3.72–4.58 cm for naive RL–PP, so on each circuit direct actuation appears the most precise of the three (supplementary Table S3). Tuned PP reaches 0.27–0.50 cm on the same circuits, and the naive bead averages 3.32 cm on its training circuits but 5.02 cm on the five unseen ones: untuned baselines reverse the ranking, and leakage inflates the learned controller.

Driving at its own pace, the bead needs about 2.3 times

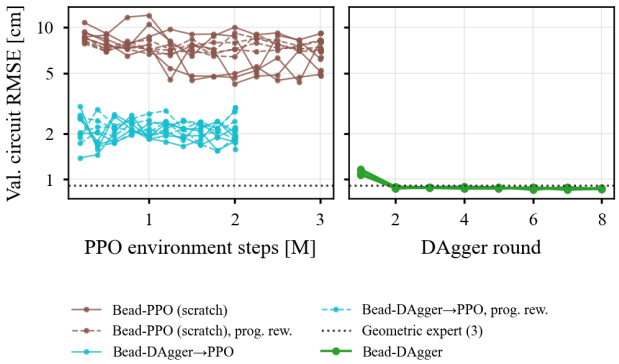


Fig. 3 RMSE on the validation circuits as training proceeds (five seeds). Left: PPO from random initial weights and PPO from the DAgger actor, with the base (solid) and progress (dashed) reward. Right: the eight DAgger rounds (expert teacher). Dotted: expert.

the car's lap time, as friction caps its speed at roughly 1.3 m/s (supplementary Fig. S5). At v_{match} the car's lap time nearly equals the bead's (42.2 versus 42.5 s) and tuned PP is still the more accurate (P1).

Dynamic vehicle model. To check whether the kinematic car flatters PP, the car controllers were also run at v_{match} on the dynamic single-track model with linear tyres of F1TENTH Gym [1] (its parameters; position at the rear axle; steering limit kept at ± 1.10 rad, as arena hairpins need about 1 rad), (a) with the tyres alone and (b) also with its steering-rate limit of 3.2 rad/s, a full-scale actuator value applied to shrunk circuits. Fixed PP and Stanley were re-tuned on each model with the same grids, references and score as before, and RL–PP was retrained (Table 5). **Gains tuned on the kinematic car do not transfer**: unchanged, Fixed PP reaches 3.95 cm in (a) and completes no lap in (b). **Re-tuned, Fixed PP is again the most accurate car controller**, at 1.83 cm in (a) and 3.09 cm in (b), about six and ten times its kinematic error, and it beats retrained RL–PP on all eight circuits (differences -3.42 cm [-4.96, -2.22] and -3.64 cm [-5.90, -2.06]). The lead over Stanley is clear with tyres only, -0.82 cm [-1.44, -0.10] and better on 7 of 8 circuits, and unclear with the rate limit, -0.49 cm [-1.22, 0.18]. These results use an extended gain range ($g \leq 6$ in (a), $g \leq 2.5$ in (b)): on the shared grid the selected gain sits on the top edge ($g = 1.4$), against our inside-the-range rule, and in (b) no setting completes every training lap. The extended optima lie inside the range and complete every test lap; the shared-grid result (Table 5) is equal within the intervals. In (b) no Stanley setting completes every training lap, so the best-scoring one was used, and it fails on Yas Marina. The bead has no tyres, so the comparison with it holds only at equal model fidelity: re-tuned PP on the tyre car (1.83 cm) is less accurate than Bead-DAgger on the tyre-free bead (1.06 cm).

Table 5 Eight test circuits at v_{match} on the dynamic single-track car: (a) tyres only, (b) tyres and steering-rate limit. Settings: (L_d, g) or (k, k_s). RMSE in cm against the extracted reference (Extr.) and the true centerline (True); Laps: % completed.

	(a) Tyres only				(b) Tyres + rate limit			
Controller	Settings	Extr.	True	Laps	Settings	Extr.	True	Laps
Fixed PP, re-tuned	(0.175, 2.5)	1.83	2.31	100	(0.45, 1.6)	3.09	3.53	100
Fixed PP, re-tuned, $g \leq 1.4$	(0.2, 1.4)	1.91	2.30	100	(0.4, 1.4)	3.23	3.59	88
Stanley, re-tuned	(8, 0.25)	2.66	3.11	100	(4, 0.05)	3.58	3.83	88
RL–PP, retrained	–	5.26	5.38	82	–	6.73	6.82	50
Fixed PP, unchanged	–	3.95	4.11	88	–	34.84	34.77	0
Stanley, unchanged	–	2.80	3.23	100	–	28.38	28.47	0
RL–PP, unchanged	–	5.68	5.71	73	–	23.52	23.39	2

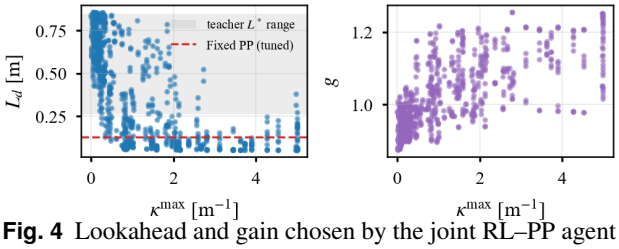


Fig. 4 Lookahead and gain chosen by the joint RL–PP agent (seed 0) while driving the unseen Montreal circuit: L_d and g versus previewed curvature. Shaded: teacher's L^* range; dashed: tuned Fixed PP.

5.5 The learned PP schedule

The RL–PP agent shortens its lookahead before corners (correlation -0.68 with the previewed curvature) and raises its gain in corners (correlation $+0.70$), as described in [4] (Fig. 4). Its median lookahead over all test circuits and seeds is, however, 0.44 m, with 57% of its steps inside the teacher's band $[0.26, 0.85]$ m, far above the 0.125 m selected for Fixed PP: its reward pulls it toward a teacher that favours long lookaheads, which explains why it is less accurate than tuned constant PP (P3). The L_d-only agent shows a small but uncertain advantage over the joint agent: it is better on 7 of 8 circuits, but the mean difference 0.26 cm has the interval $[-0.30, 0.87]$. We did not redesign the teacher, which would tune the method of [4] beyond adapting it.

5.6 Statistical evidence

In all four primary comparisons (Table 6) the same controller is better on all eight circuits, so the Wilcoxon test gives its smallest attainable value, $p = 0.0078$, which remains below 0.05 after Holm correction ($p = 0.031$). With eight circuits this only says that all circuits agree; effect sizes are given by the intervals, which describe variation over these circuits and seeds, not over other simulators, scales or vehicles.

5.7 Threats to validity

- **Extraction error and spacing.** Differences below about 1 cm are within the extraction error; Table 4 repeats the key results against the true centerlines. The Austin test reference does not close exactly; closing it

Table 6 Pairwise RMSE differences A−B in cm across all eight test circuits (negative favours A; 95% bootstrap interval over circuits and training seeds; exact Wilcoxon test). Left: against the extracted references. Right: the same comparison against the true F1TENTH centerlines, recomputed from the stored trajectories.

	Extracted reference					True centerline	
A vs B	Diff.	95% CI	Better	p	Holm	Diff.	95% CI
Primary (Holm over these four)							
DAgger vs Fixed PP @v_{match}	+0.75	[+0.62, +0.87]	0/8	0.0078	0.031	+0.36	[+0.22, +0.51]
DAgger→PPO vs DAgger	+0.71	[+0.53, +0.93]	0/8	0.0078	0.031	+0.66	[+0.39, +0.97]
RL–PP joint vs Fixed PP	+1.49	[+1.06, +2.02]	0/8	0.0078	0.031	+0.97	[+0.55, +1.49]
Bead-PP vs Expert	−0.79	[−0.92, −0.64]	8/8	0.0078	0.031	−0.36	[−0.49, −0.23]
Exploratory (no correction)							
DAgger vs Expert	−0.01	[−0.06, +0.04]	5/8	0.8438	–	−0.00	[−0.04, +0.04]
DAgger (Bead-PP teacher) vs DAgger	−0.64	[−0.77, −0.48]	8/8	0.0078	–	−0.36	[−0.52, −0.19]
DAgger (Bead-PP teacher) vs Bead-PP	+0.15	[+0.11, +0.18]	0/8	0.0078	–	+0.00	[−0.03, +0.04]
DAgger (Bead-PP teacher) vs Fixed PP @v_{match}	+0.12	[+0.08, +0.15]	0/8	0.0078	–	+0.01	[−0.03, +0.05]
DAgger→PPO ($\lambda = 10$) vs DAgger	+0.20	[+0.15, +0.26]	0/8	0.0078	–	+0.13	[+0.05, +0.21]
DAgger→PPO ($\lambda = 10$) vs DAgger→PPO	−0.50	[−0.75, −0.31]	8/8	0.0078	–	−0.53	[−0.84, −0.24]
DAgger→PPO vs Expert	+0.69	[+0.53, +0.94]	0/8	0.0078	–	+0.66	[+0.41, +0.99]
PPO (scratch) vs DAgger→PPO	+3.36	[+2.35, +4.92]	0/8	0.0078	–	+3.23	[+2.23, +4.83]
DAgger→PPO (prog.) vs DAgger→PPO	+0.37	[+0.08, +0.64]	0/8	0.0078	–	+0.30	[−0.05, +0.66]
PPO (scratch, prog.) vs PPO (scratch)	+1.37	[+0.19, +2.33]	0/8	0.0078	–	+1.38	[+0.17, +2.42]
Bead-PP vs Fixed PP @v_{match}	−0.03	[−0.08, +0.03]	5/8	0.5469	–	+0.01	[−0.01, +0.02]
Fixed PP @v_{match} vs Expert	−0.76	[−0.86, −0.66]	8/8	0.0078	–	−0.36	[−0.49, −0.25]
RL–PP joint vs RL–PP L_d	+0.26	[−0.30, +0.87]	1/8	0.0234	–	+0.27	[−0.24, +0.82]
Adaptive PP vs Fixed PP	+0.08	[+0.06, +0.11]	0/8	0.0078	–	+0.03	[+0.02, +0.05]
Stanley vs Fixed PP	+3.78	[+3.45, +4.05]	0/8	0.0078	–	+3.19	[+2.88, +3.44]
Stanley @v_{match} vs Fixed PP @v_{match}	+3.25	[+2.88, +3.54]	0/8	0.0078	–	+2.67	[+2.31, +2.97]

changes RMSE by at most 0.012 cm and no lap outcome. Waypoints are 11.1–23.5 cm apart, which may disadvantage the expert, whose target is the next waypoint, whereas PP interpolates its lookahead point.

- **PPO budget and settings.** 2–3 × 10^6 steps per run; of the fine-tuning settings only the behaviour-cloning weight was varied ($\lambda \in \{0.1, 1, 10\}$, three seeds), while initial noise, learning rate and critic warm-up were fixed. Our results show that PPO did not improve the policy under these settings, not that it cannot.
- **RL–PP adaptation.** Scale, ranges, teacher, reward weights and budget differ from [4] (Table 2); results characterize our adaptation.
- **Plants and scale.** Bead-PP controls for the plant for one law and one speed; the main comparison uses a kinematic car; on a dynamic single-track model the error of re-tuned PP grows from 0.31 to 1.83 cm at v_{match}, and under the steering-rate limit its gain range had to be extended for any setting to complete every training lap; tracks are shrunk 4.3–9.8×.
- **Exposure and scope.** Three test circuits were known when the reward was designed, and a preliminary run touched the test circuits (Section 4.1); five seeds; only 50 of the 150 test contours in each synthetic set; simulation only. Not yet tested: sensor noise and true-scale circuits with real steering limits.

6 Conclusion

Under shared paths, measures and lap rule, we compared learning that replaces a geometric tracking law with learning integrated into it, on held-out circuits. The DAgger-trained bead matched its hand-written teacher; taught by the PP law, it came within 0.15 cm of that law on the extracted references and matched it, and tuned PP on the car, against the true centerlines, so the teacher, more than the network, limited imitation. With our PPO budget and settings, PPO did not improve the imitation policy, whether started from scratch or fine-tuned, with either reward and with any of the behaviour-cloning weights tried. A tuned PP law was more accurate than every learned controller that did not imitate it, on the kinematic car and on the bead, so the gap comes from the control law rather than the vehicle; it shrinks to 0.36 cm against the true centerlines but keeps its sign on every circuit. Our adaptation of RL-tuned PP reproduced the qualitative schedule of [4] but followed a teacher with long lookaheads. A naive pro-

tocol would have reversed these conclusions. A well-tuned PP law is thus a strong baseline for learned trackers (tuned Stanley, which regulates the front axle, less so); on a dynamic car it stays the most accurate car controller only after re-tuning, with a smaller margin, so baselines should be tuned on the vehicle model used for comparison; integrating learning with it—for example a learned residual on top of PP, or better-designed RL tuning of its parameters—is a more promising direction than replacing it, together with dynamic vehicle models with tyre slip and hardware tests.

Author Contributions [To be completed by the authors: the contributions of Dileep Banothu and N. Sanjay Varma.]

Funding Not applicable.

Data Availability The code, data split, trained checkpoints, raw test trajectories and the scripts that generate every table and number of this paper are available at `https://github.com/sanjayvarma06/Bead-Simluation-using-RL-`. The F1TENTH centerlines are available at `https://github.com/f1tenth/f1tenth_racetracks`.

Declarations

Conflict of Interest The authors declare that there is no competing financial interest or personal relationship that could have appeared to influence the work reported in this paper.

References

1. M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1TENTH: An open-source evaluation environment for continuous control and reinforcement learning," *Proc. of the NeurIPS 2019 Competition and Demonstration Track*, PMLR, vol. 123, pp. 77-89, 2020.
2. R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Technical Report CMU-RI-TR-92-01, Robotics Institute, Carnegie Mellon University, 1992.
3. J. M. Snider, "Automatic steering methods for autonomous automobile path tracking," Technical Report CMU-RI-TR-09-08, Robotics Institute, Carnegie Mellon University, 2009.
4. M. Elgouhary and A. S. El-Wakeel, "Learning to tune pure pursuit in autonomous racing: Joint lookahead and steering-gain control with PPO," *arXiv preprint* arXiv:2602.18386, 2026.
5. J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint* arXiv:1707.06347, 2017.
6. S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," *Proc. of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 627-635, 2011.
7. G. M. Hoffmann, C. J. Tomlin, M. Montemerlo, and S. Thrun, "Autonomous automobile trajectory tracking for off-road driving: Controller design, experimental validation and racing," *Proc. of the American Control Conference*, pp. 2296-2301, 2007.
8. R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed., Springer, New York, 2012.
9. J. Kong, M. Pfeiffer, G. Schildbach, and F. Borrelli, "Kinematic and dynamic vehicle models for autonomous driving control design," *Proc. of the IEEE Intelligent Vehicles Symposium*, pp. 1094-1099, 2015.
10. A. Heilmeier, A. Wischnewski, L. Hermansdorfer, J. Betz, M. Lienkamp, and B. Lohmann, "Minimum curvature trajectory planning and control for an autonomous race car," *Vehicle System Dynamics*, vol. 58, no. 10, pp. 1497-1527, 2020.
11. J. Betz, H. Zheng, A. Liniger, U. Rosolia, P. Karle, M. Behl, V. Krovi, and R. Mangharam, "Autonomous vehicles on the edge: A survey on autonomous vehicle racing," *IEEE Open Journal of Intelligent Transportation Systems*, vol. 3, pp. 458-488, 2022.
12. B. R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A. A. Al Sallab, S. Yogamani, and P. Pérez, "Deep reinforcement learning for autonomous driving: A survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 6, pp. 4909-4926, 2022.
13. J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *International Journal of Robotics Research*, vol. 32, no. 11, pp. 1238-1274, 2013.
14. J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *Proc. of the International Conference on Learning Representations (ICLR)*, 2016.
15. M. Andrychowicz, A. Raichuk, P. Stańczyk, M. Orsini, S. Girgin, R. Marinier, L. Hussenot, M. Geist, O. Pietquin, M. Michalski, S. Gelly, and O. Bachem, "What matters in on-policy reinforcement learning? A large-scale empirical study," *Proc. of the International Conference on Learning Representations (ICLR)*, 2021.
16. T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," *Proc. of the 35th International Conference on Machine Learning (ICML)*, pp. 1861-1870, 2018.
17. B. D. Evans, H. W. Jordaan, and H. A. Engelbrecht, "Comparing deep reinforcement learning architectures for autonomous racing," *Machine Learning with Applications*, vol. 14, 100496, 2023.
18. D. A. Pomerleau, "ALVINN: An autonomous land vehicle in a neural network," *Advances in Neural Information Processing Systems 1*, pp. 305-313, 1989.
19. N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62-66, 1979.
20. T. Y. Zhang and C. Y. Suen, "A fast parallel algorithm for thinning digital patterns," *Communications of the ACM*, vol. 27, no. 3, pp. 236-239, 1984.
21. A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1-8, 2021.